

Lin Deming - Project Portfolio

PROJECT: ModsUni

1. Overview

modsUni is a desktop timetable management application created by students for students. It incorporates a simple GUI with majority of its user interactions occurring in its in-built CLI.

2. About this portfolio

This project portfolio documents my contributions to the ModsUni project. It includes a summary of features that I implemented and other contributions to the project.

3. Summary of contributions

- **Major enhancement:** This enhancement added functionality to **manage modules in the database**.
 - What it does: The add module feature adds a non-existing module to the database. Conversely, the remove module feature removes a existing module from the database. In addition, the edit module feature allows modification of the information of existing modules.
 - Justification: When there is a change in module information, introduction of new modules or removal of modules, the database should match the changes. The administrator will then have to do modify the database to keep it up to date. The `addModuleDB`, `removeModuleDB` and `editModule` command allows the administrator to perform such tasks.
 - Highlights: This enhancement is an end-to-end feature and makes use of almost all components of the software. Though the implementation of these features is intuitive and makes use of existing code wherever possible, it was challenging as it required the creation of a `PrereqGenerator` class (to handle parsing) from scratch.
 - Relevant pull requests: [#92](https://github.com/CS2103-AY1819S1-W17-2/main/pull/92) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/92>), [#123](https://github.com/CS2103-AY1819S1-W17-2/main/pull/123) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/123>) and [#186](https://github.com/CS2103-AY1819S1-W17-2/main/pull/186) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/186>)
- **Minor enhancement:** This enhancement added support for multiple administrator accounts.
 - What it does: This enhancement allows logged in admin users to create new admin accounts. Similarly, they can choose to remove other admin users as well.
 - Justification: Since the database have a large number of modules, administrator may find it difficult to manage the entire database by alone. Having the ability to assign new administrators to help spread the workload makes it much easier to manage the database.
 - Highlights: This enhancement is an end-to-end feature and makes use of almost all components of the software, from `Storage` to `Ui`. It was challenging as the feature had to handle validation of different data types, to ensure that the feature functions sensibly.
 - Relevant pull requests: [#46](https://github.com/CS2103-AY1819S1-W17-2/main/pull/46) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/46>), [#169](https://github.com/CS2103-AY1819S1-W17-2/main/pull/169) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/169>) and [#186](https://github.com/CS2103-AY1819S1-W17-2/main/pull/186) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/186>)

- Code contributed: [\[Functional code](https://nus-cs2103-ay1819s1.github.io/cs2103-dashboard/#=undefined&search=deming16&sort=displayName&since=2018-09-12&until=2018-11-07&timeframe=day&reverse=false&repoSort=true)
(<https://nus-cs2103-ay1819s1.github.io/cs2103-dashboard/#=undefined&search=deming16&sort=displayName&since=2018-09-12&until=2018-11-07&timeframe=day&reverse=false&repoSort=true>)
]
- Other contributions:
 - Project management on GitHub:
 - Updated descriptions and labels of PRs (where necessary)
 - Organised and assigned issues
 - Enhancements to existing features:
 - Refactored some existing `Person` fields to fit the new `User` fields (Pull request [#46](https://github.com/CS2103-AY1819S1-W17-2/main/pull/46) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/46>))
 - Updated existing test cases to fit new code (Pull request [#92](https://github.com/CS2103-AY1819S1-W17-2/main/pull/92) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/92>))
 - Documentation:
 - Updated the User Guide (Pull requests [#55](https://github.com/CS2103-AY1819S1-W17-2/main/pull/55) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/55>), [#128](https://github.com/CS2103-AY1819S1-W17-2/main/pull/128) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/128>), [#204](https://github.com/CS2103-AY1819S1-W17-2/main/pull/204) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/204>) and [#267](https://github.com/CS2103-AY1819S1-W17-2/main/pull/267) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/267>))
 - Updated the Developer Guide (Pull requests [#128](https://github.com/CS2103-AY1819S1-W17-2/main/pull/128) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/128>), [#142](https://github.com/CS2103-AY1819S1-W17-2/main/pull/142) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/142>) and [#204](https://github.com/CS2103-AY1819S1-W17-2/main/pull/204) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/204>))
 - Community:
 - Reported bugs for other teams in the class ([#1](https://github.com/CS2103-AY1819S1-T12-1/main/issues/147) (<https://github.com/CS2103-AY1819S1-T12-1/main/issues/147>), [#2](https://github.com/CS2103-AY1819S1-T12-1/main/issues/153) (<https://github.com/CS2103-AY1819S1-T12-1/main/issues/153>), [#3](https://github.com/CS2103-AY1819S1-T12-1/main/issues/161) (<https://github.com/CS2103-AY1819S1-T12-1/main/issues/161>) and [#4](https://github.com/CS2103-AY1819S1-T12-1/main/issues/135) (<https://github.com/CS2103-AY1819S1-T12-1/main/issues/135>))
 - Reviewed PRs (with non-trivial review comments): [#131](https://github.com/CS2103-AY1819S1-W17-2/main/pull/131) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/131>) and [#184](https://github.com/CS2103-AY1819S1-W17-2/main/pull/184) (<https://github.com/CS2103-AY1819S1-W17-2/main/pull/184>))

4. Contributions to the User Guide

Given below are sections I contributed to the User Guide. They showcase my ability to write documentation targeting end-users.

Add a new module to database: `addModuleDB`

Creates a new module and add it to the module database.

Format: `addModuleDB OPTION/INPUT...`

Input fields:

- Module code : `code/<MODULE_CODE>` — It should contain only alphanumeric characters.
- Department : `department/<DEPARTMENT>`
- Title : `title/<TITLE>`
- Description : `description/<DESCRIPTION>`
- Credit : `credit/<CREDIT>`

- Available semesters : `available/[SEM1][SEM2][SPECIAL_TERM1][SPECIAL_TERM2]`

NOTE

For each entry enter '1' if the module will be available. Otherwise, enter '0'.
Only the first four digits will be taken into account and if there are less than four digits the remaining input will be '0' by default.

- Prerequisites : `prereq/<PREREQ_STRING>`

NOTE

Format for PREREQ_STRING: `<&OR|>[<MODULE_CODE,>[MORE_MODULE_CODE,]][BRANCH][MORE_BRANCH]`
Format for BRANCH: `(<&OR|>[<MODULE_CODE,>[MORE_MODULE_CODE,]][BRANCH][MORE_BRANCH])`
To represent no prerequisite, use an empty string or a string that consist of only '&' or '|'.
To represent only 1 prerequisite, use '&' or '|' followed by the module code.

Example:

- `addModuleDB code/CS2103T department/Computing title/The New Module credit/4 available/1100 description/This course is newly offered by the school of computing. Visit the module website for more details prereq/|(|CS1020,CS1020E,CS2020,)(&CS2030,(|CS2040,CS2040C,))`
Create and add a new module with module code "CS2103T" and the stated details to the module database.

Edit details of a module in database: `editModule`

Edit the details of a module specified by its index in database panel in database.

Format: `editModule <INDEX> OPTION/INPUT...`

Input fields:

- Module code : `code/[MODULE_CODE]` — It should contain only alphanumeric characters.
- Department : `department/[DEPARTMENT]`
- Title : `title/[TITLE]`
- Description : `description/[DESCRIPTION]`
- Credit : `credit/[CREDIT]`
- Available semesters : `available/[SEM1][SEM2][SPECIAL_TERM1][SPECIAL_TERM2]`

NOTE

For each entry enter '1' if the module will be available. Otherwise, enter '0'.
Only the first four digits will be taken into account and if there are less than four digits the remaining input will be '0' by default.

- Prerequisites : `prereq/[PREREQ_STRING]`

NOTE

Format for PREREQ_STRING: `<&OR|>[<MODULE_CODE,>[MORE_MODULE_CODE,]][BRANCH][MORE_BRANCH]`
Format for BRANCH: `(<&OR|>[<MODULE_CODE,>[MORE_MODULE_CODE,]][BRANCH][MORE_BRANCH])`
To represent no prerequisite, use an empty string or a string that consist of only '&' or '|'.
To represent only 1 prerequisite, use '&' or '|' followed by the module code.

Example:

- `editModule 8 code/CS2103 credit/4`

Change the module code and credit of module with index "8" to "CS2103" and "4" respectively.

NOTE

After editing, the database panel will display the updated list as shown below.

User	Staged	Taken	Database
4. CS1231			Computer Science Discrete Structures 4 MCs
5. CS1010			Computer Science Programming Methodology 4 MCs
6. CS2040			Computer Science Data Structures and Algorithms 4 MCs
7. CS2103T			Computer Science Software Engineering 4 MCs
8. CS2109			Sem1 Sem2 SOC The new module 5 MCs

Command: `editModule 8 code/CS2103 credit/4`



User	Staged	Taken	Database
4. CS1231			Computer Science Discrete Structures 4 MCs
5. CS1010			Computer Science Programming Methodology 4 MCs
6. CS2040			Computer Science Data Structures and Algorithms 4 MCs
7. CS2103T			Computer Science Software Engineering 4 MCs
8. CS2103			Sem1 Sem2 SOC The new module 4 MCs

Remove a existing module from database: `removeModuleDB`

Removes a existing module from the module database.

Format: `removeModuleDB <MODULE_CODE>`

Example:

- `removeModuleDB CS2109`

Removes a module with module code "CS2109" if it exists.

NOTE

After removing, the database panel will display the updated list as shown below.

User	Staged	Taken	Database
1.			CS2030 Computer Science Programming Methodology II 4 MCs
2.			CS1231 Computer Science Discrete Structures 4 MCs
3.			CS1010 Computer Science Programming Methodology 4 MCs
4.			CS2040 Computer Science Data Structures and Algorithms 4 MCs
5.			CS2109 Computer Science Software Engineering 4 MCs

Command: `removeModuleDB CS2109`



User	Staged	Taken	Database
1.			CS2030 Computer Science Programming Methodology II 4 MCs
2.			CS1231 Computer Science Discrete Structures 4 MCs
3.			CS1010 Computer Science Programming Methodology 4 MCs
4.			CS2040 Computer Science Data Structures and Algorithms 4 MCs

5. Contributions to the Developer Guide

Given below are sections I contributed to the Developer Guide. They showcase my ability to write technical documentation and the technical depth of my contributions to the project.

Module Database Administration

Module database administration is a series of commands that administrators can use to make necessary changes to keep the module database up to date, or to cater to their own needs.

NOTE

The commands pertaining to module database administration can only be executed by users of type Admin .

Administrators can choose to add, remove or edit a module using addModuleDB , removeModuleDB and editModule respectively. The changes will then be reflected in both the module list in memory and in storage. Figure 1(shown below) describes the general flow of the administration of the module database in an activity diagram.

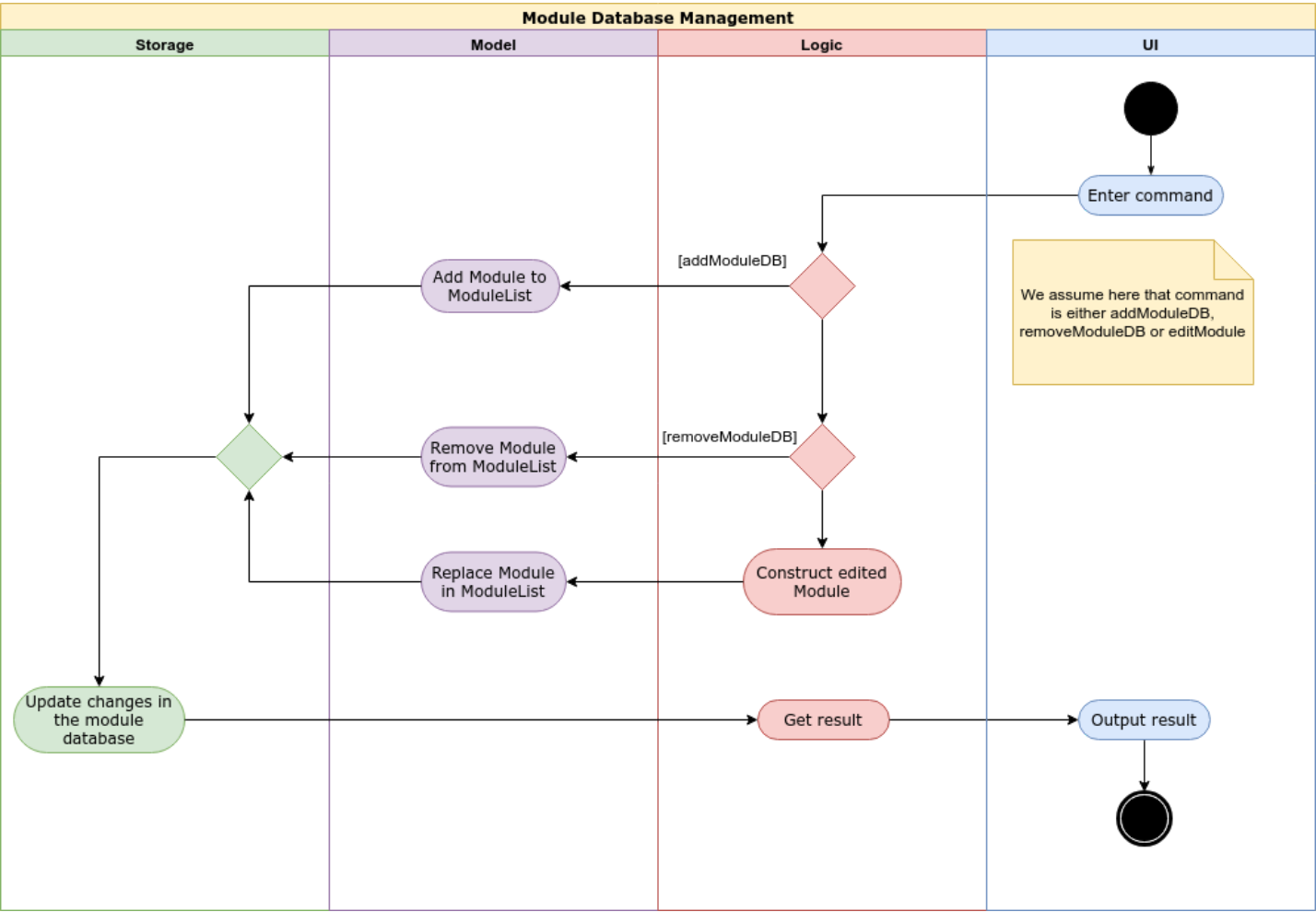


Figure 1. Activity diagram for module database administration.

Adding module to database

`addModuleDB` mechanism is facilitated by `AddModuleToDatabaseCommand`. It allows the addition of modules to database.

The `AddModuleToDatabaseCommand` extends for `Command`. Figure 2(shown below) depicts the UML representation of the `AddModuleToDatabaseCommand`.

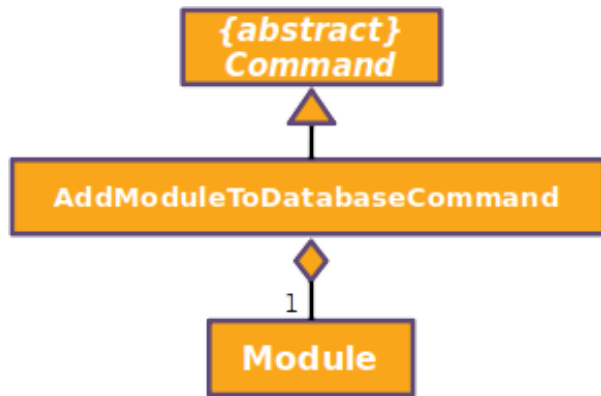


Figure 2. UML diagram of `AddModuleToDatabaseCommand`.

Parsing of command is performed by `AddModuleToDatabaseCommandParser`, which returns a `AddModuleToDatabaseCommand` object after parsing `ModuleCode`, `Title`, `Credit`, `Department`, `Description`, `AvailableSem` and `Prereq`. Figure 3(shown below) shows the sequence diagram of the `AddModuleToDatabaseCommandParser`.

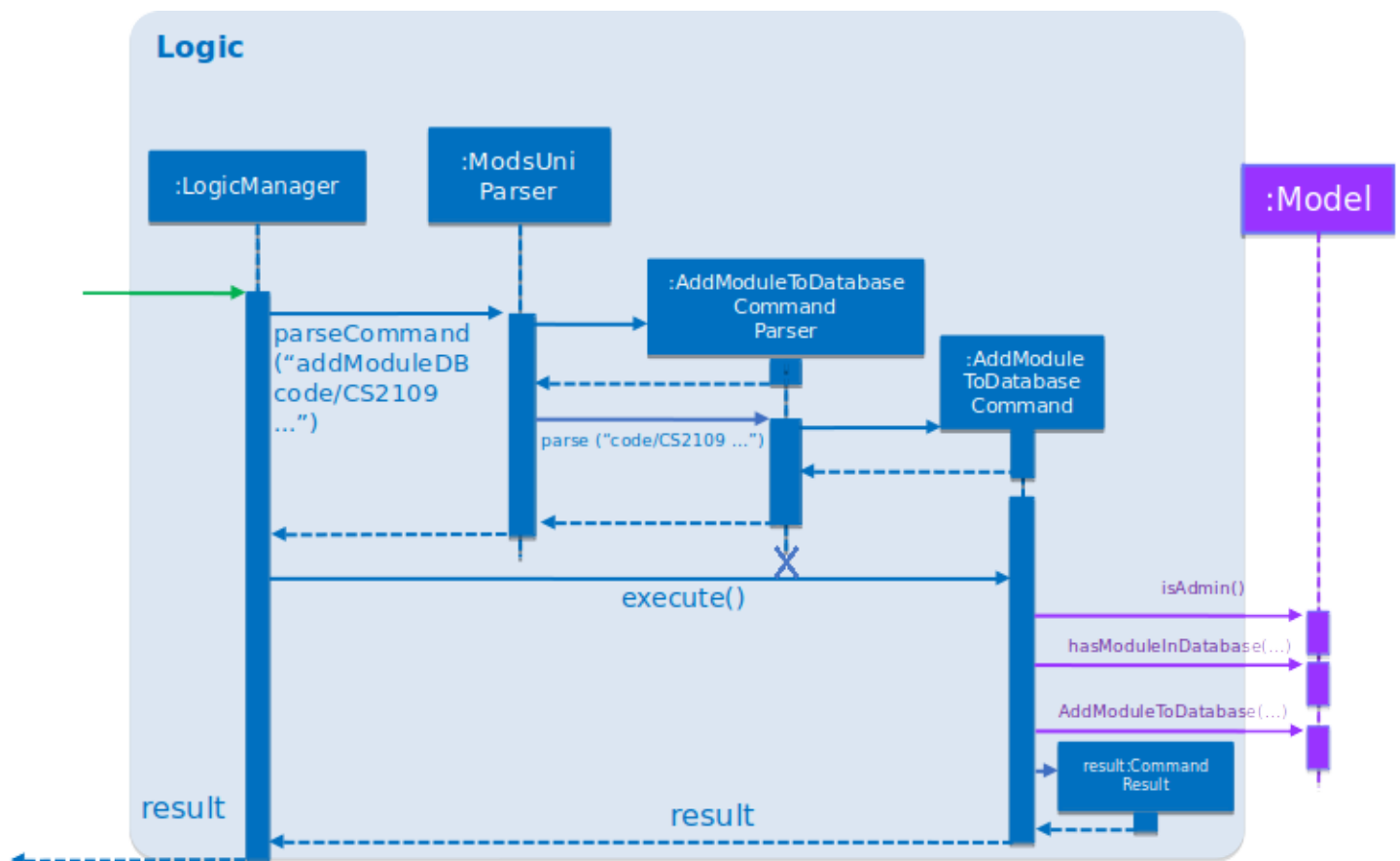


Figure 3. Sequence diagram for the interaction between **Logic** and **Model** Components when executing `AddModuleToDatabaseCommand`.

Figure 4(shown below) shows the high-level sequence diagram of the command execution.

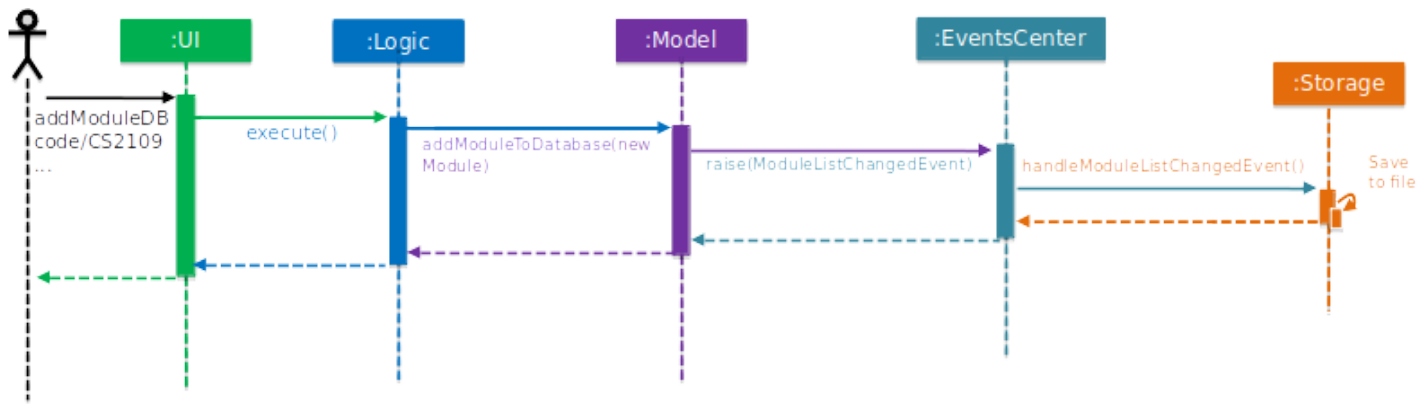


Figure 4. High-Level Sequence diagram of adding a module to database.

Given below is an example usage scenario and how the `addModuleDB` mechanism behaves at each step:

NOTE

For this example usage scenario, we assume that module "CS2109" did not exist in the database initially.

1. An admin executes `addAdminDB` with valid arguments to add a module with module code "CS2109" to the module list. The `addModuleDB` command calls `Model#addModuleToDatabase(...)`, causing the new model to be added to `ModuleList` and subsequently update the `moduleList.xml` file with the new list.
2. Another admin did not know the new module has already been added and tries to add the "CS2109" again by executing `addModuleDB`. The `addModuleDB` command calls `Model#hasModuleInDatabase(...)`, causing a feedback to return, informing the admin that the module already exists, and the module is not added.

The code snippet below shows how user input is parsed into a `AddModuleToDatabaseCommand` object.

```

public AddModuleToDatabaseCommand parse(String args) throws ParseException {
    ArgumentMultimap argMultimap =
        ArgumentTokenizer.tokenize(args, PREFIX_MODULE_CODE, PREFIX_MODULE_DEPARTMENT,
PREFIX_MODULE_TITLE,
        PREFIX_MODULE_DESCRIPTION, PREFIX_MODULE_CREDIT, PREFIX_MODULE_AVAILABLE,
PREFIX_MODULE_PREREQ);

    if (!arePrefixesPresent(argMultimap, PREFIX_MODULE_CODE, PREFIX_MODULE_DEPARTMENT, PREFIX_MODULE_TITLE,
        PREFIX_MODULE_DESCRIPTION, PREFIX_MODULE_CREDIT, PREFIX_MODULE_AVAILABLE, PREFIX_MODULE_PREREQ)
        || !argMultimap.getPreamble().isEmpty()) {
        throw new ParseException(String.format(MESSAGE_INVALID_COMMAND_FORMAT,
            AddModuleToDatabaseCommand.MESSAGE_USAGE));
    }

    String codeName = argMultimap.getValue(PREFIX_MODULE_CODE).get();
    Code code = new Code(codeName);
    String department = argMultimap.getValue(PREFIX_MODULE_DEPARTMENT).get();
    String title = argMultimap.getValue(PREFIX_MODULE_TITLE).get();
    String description = argMultimap.getValue(PREFIX_MODULE_DESCRIPTION).get();
    int credit = Integer.parseInt(argMultimap.getValue(PREFIX_MODULE_CREDIT).get());
    boolean[] sems = getAvailableSems(argMultimap.getValue(PREFIX_MODULE_AVAILABLE).get());
    Prereq prereq = ParserUtil.parsePrereq(argMultimap.getValue(PREFIX_MODULE_PREREQ).get());

    Module module = new Module(code, department, title, description, credit,
        sems[0], sems[1], sems[2], sems[3], new ArrayList<Code>(), prereq);

    return new AddModuleToDatabaseCommand(module);
}

```

JAVA

The created `Command` object then execute the command, shown in the code snippet below.

```
public CommandResult execute(Model model, CommandHistory history) throws CommandException {
    requireNonNull(model);
    if (model.getCurrentUser() == null) {
        throw new CommandException(MESSAGE_NOT_LOGGED_IN);
    }

    if (!model.isAdmin()) {
        throw new CommandException(MESSAGE_NOT_ADMIN);
    }

    if (model.hasModuleInDatabase(toAdd)) {
        throw new CommandException(MESSAGE_DUPLICATE_MODULE);
    }

    model.addModuleToDatabase(toAdd);

    return new CommandResult(String.format(MESSAGE_SUCCESS));
}
```

Deleting module from database

removeModuleDB mechanism is facilitated by the RemoveModuleFromDatabaseCommand class. It allows Admin to remove Module from the database.

The RemoveModuleFromDatabaseCommand extends for Command . Figure 5(shown below) depicts the UML representation of the RemoveModuleFromDatabaseCommand .

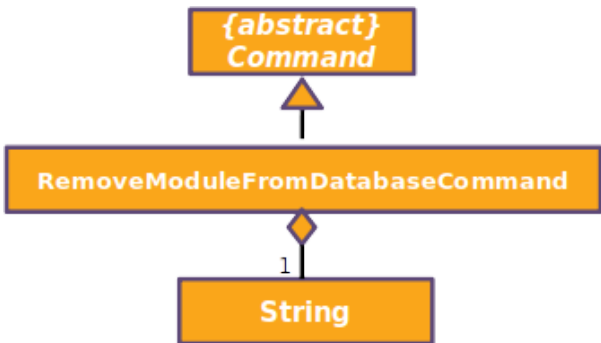


Figure 5. UML diagram of RemoveModuleFromDatabaseCommand .

Parsing of command is performed by RemoveModuleFromDatabaseParser , which returns a RemoveModuleFromDatabase object after parsing moduleCode object. Figure 6(shown below) shows the sequence diagram of the RemoveModuleFromDatabaseCommandParser .

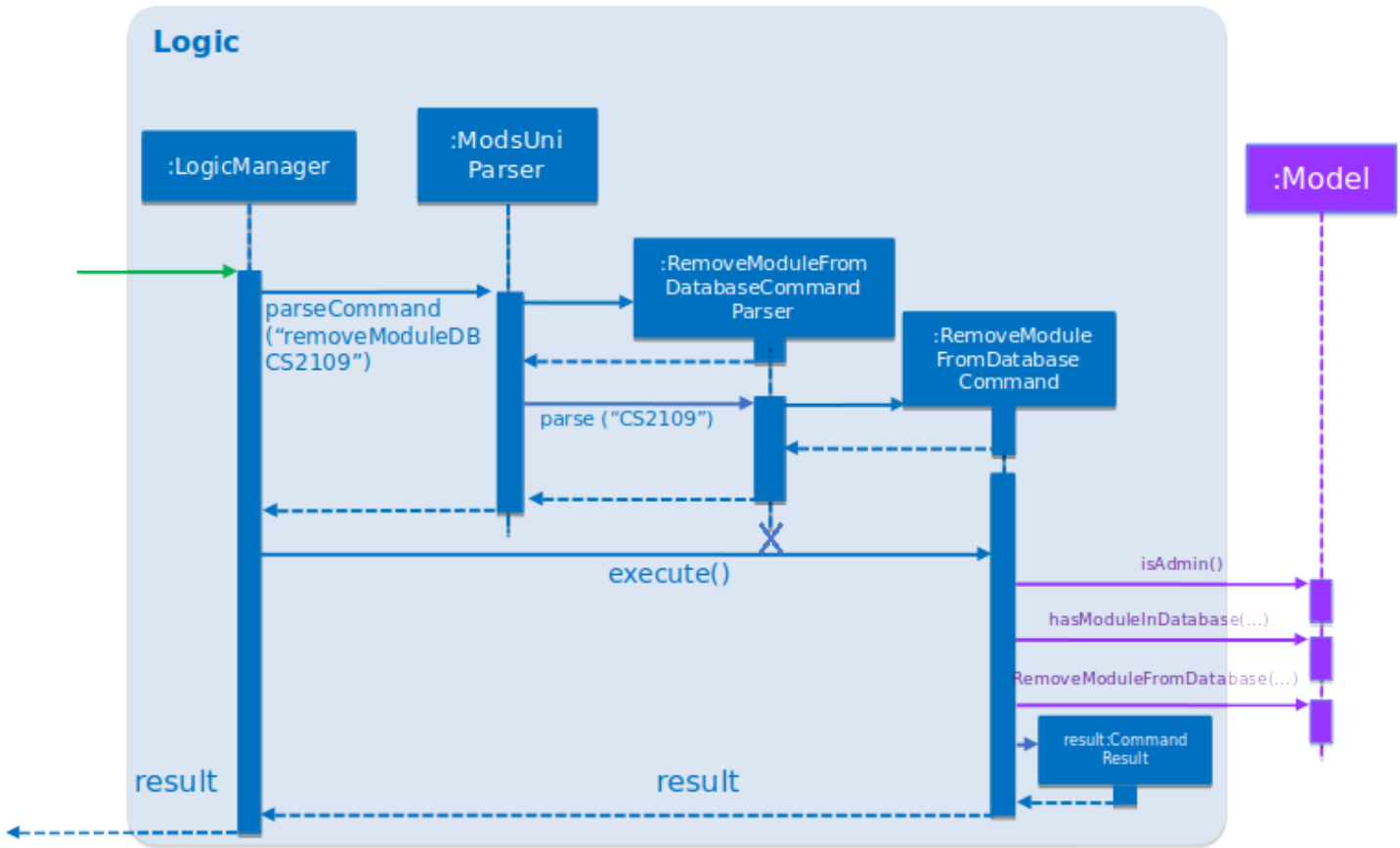


Figure 6. Sequence diagram for the interaction between Logic and Model Components when executing RemoveModuleFromDatabase .

Figure 7(shown below) shows the high-level sequence diagram of the command execution.

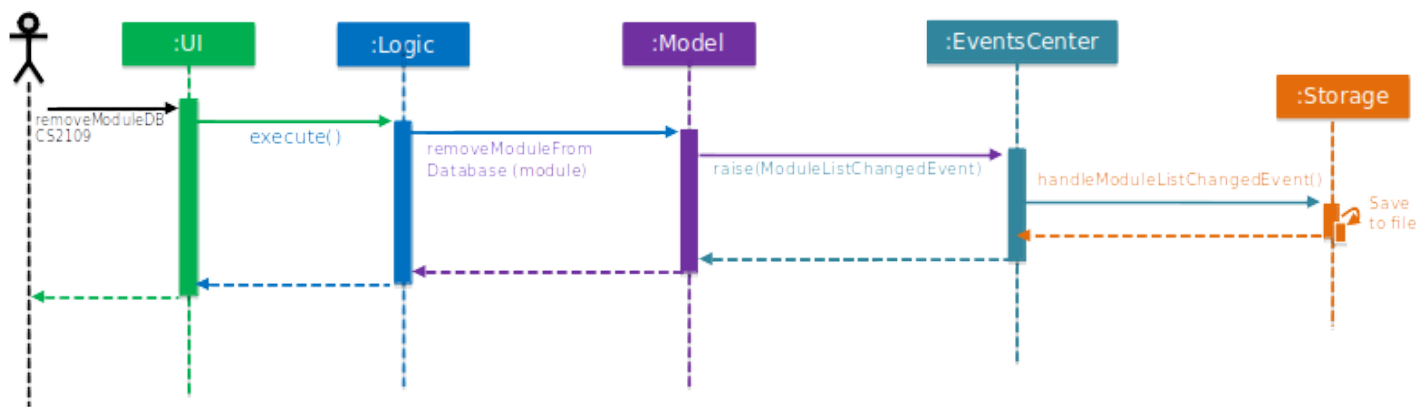


Figure 7. High-Level sequence diagram of removing a module from database.

Given below is an example usage scenario and how the `removeModuleDB` mechanism behaves at each step:

NOTE For this example usage scenario, we assume that module "CS2109" already existed.

1. An admin executes `removeAdminDB CS2109` to remove the module with module code "CS2109" from the module list. The `removeModuleDB` command calls `Model#removeModuleFromDatabase(...)`, causing the module with module code "CS2109" to be removed from the module list and subsequently update the `moduleList.xml` file with the new list.
2. Another admin did not know that module "CS2109" has already been removed and tries to remove it again by executing `removeModuleDB CS2109`. The `removeModuleDB` command returns a feedback informing the admin that the module "CS2109" does not exist, and no changes is made to `ModuleList` and `moduleList.xml`.

Editing module from database

`editModule` mechanism is facilitated by the `EditModuleCommand` class. It allows Admin to edit Module from the database.

The `EditModuleCommand` extends for `Command`. Figure 8(shown below) depicts the UML representation of the `EditModuleCommand`.

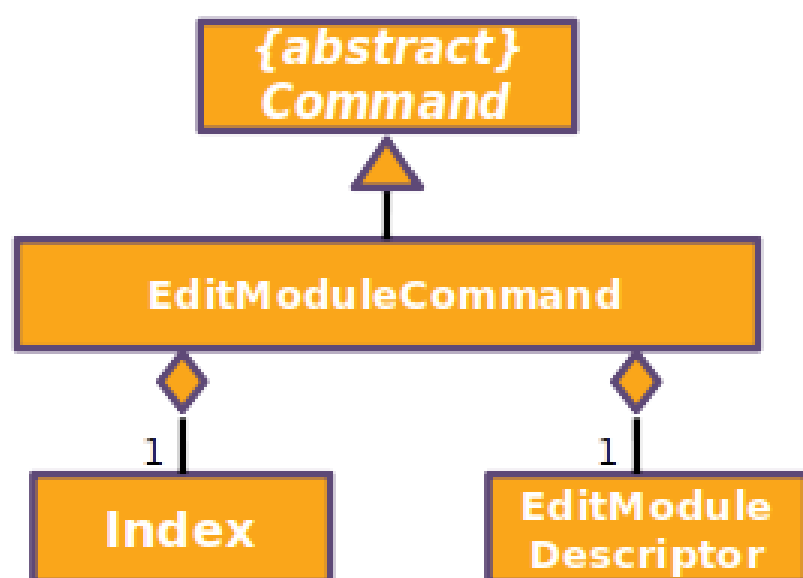


Figure 8. UML diagram of `EditModuleCommand`.

Parsing of command is performed by the `EditModuleCommandParser` , which returns a `EditModuleCommand` object after parsing `ModuleCode`, `Title`, `Credit`, `Department`, `Description`, `AvailableSem` and `Prereq`. Figure 9(shown below) shows the sequence diagram of the `EditModuleCommandParser` .

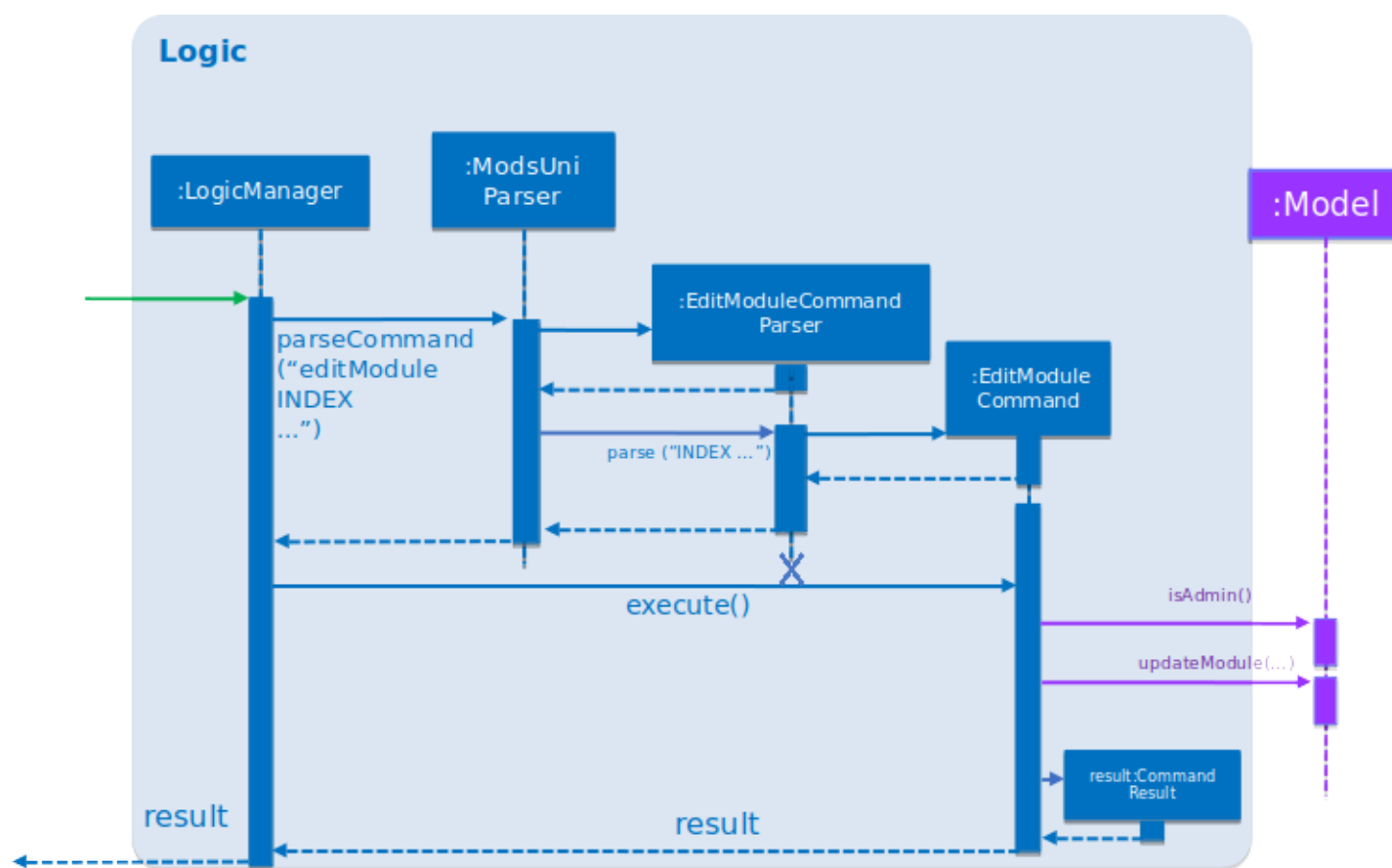


Figure 9. Sequence diagram for the interaction between Logic and Model Components when executing `EditModule` .

Figure 10(shown below) shows the high-level sequence diagram of the command execution.

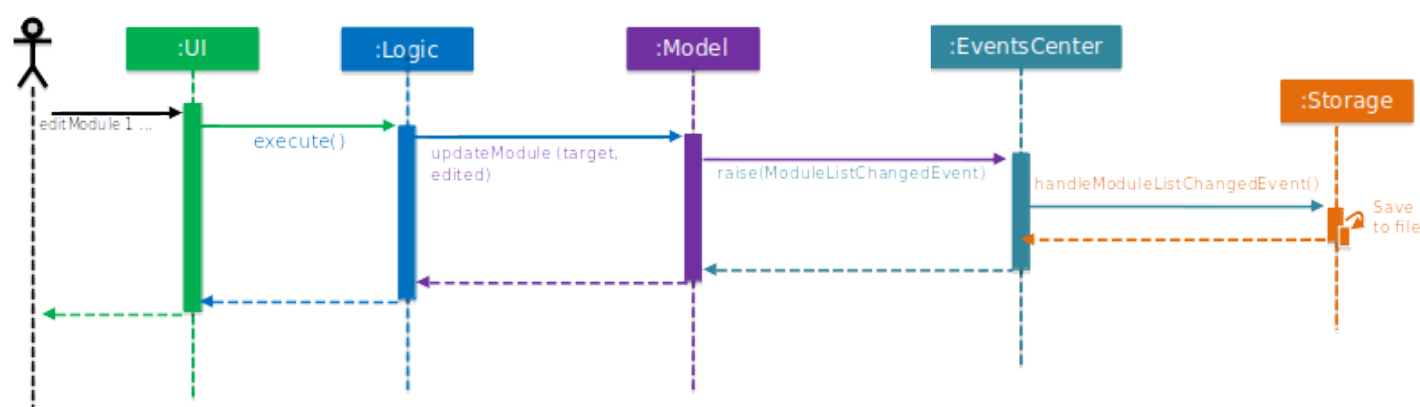


Figure 10. High-level sequence diagram of editing a module from database.

Design Considerations

Aspect: How add & remove executes

- **Alternative 1 (current choice):** Interact with `moduleList` loaded from `moduleList.xml` .
 - Pros: It is easy to implement.
 - Cons: There may be performance issues in terms of memory usage.
- **Alternative 2:** Interact directly with `moduleList.xml`

- Pros: There will be less memory used for storing module list.
- Cons: Students may experience performance issues as they have to read from `moduleList.xml` in the hard disk every time they execute command that requires reading the module list.

Aspect: Data structure for module list

- **Alternative 1 (current choice):** Using a list of `Module` .
 - Pros: We can simply store `Module` objects directly into the list.
 - Cons: We have to iterate through the list to check if a module exist, resulting in a linear time complexity.
- **Alternative 2:** Using a hash map with `ModuleCode` key and `Module` value.
 - Pros: We can simply check if a `ModuleCode` key exists, which is fast as it is in constant time.
 - Cons: Each module can only have one module code. Thus if a module have multiple module codes, we have to create another module just to store the other module codes. This can be heavy on memory usage.

Last updated 2019-08-01 16:10:40 +08